

P2645R1

path_view: a design that took a wrong turn

Published Proposal, 2024-10-10

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Changes from R0

3 Problems

3.1 Encoding

3.2 Implementation and usage experience

3.3 Performance

3.4 Formatting and output

3.5 Complexity

3.6 Conclusion

References

Informative References

"Your scientists were so preoccupied with whether or not they could, they didn't stop to think if they should."

§ 1. Introduction

P1030 `std::filesystem::path_view` is a paper with a long and troubled history, consistently falling short of its original goals and, in some cases, even regressing. While recent revisions have removed some of the more questionable parts of the design, such as the use of locales, numerous critical issues remain unresolved. This paper highlights some of these issues and argues that standardizing `path_view` in its current form would not only perpetuate past design flaws but also make future fixes nearly impossible. Additionally, it points out the severe lack of implementation and practical usage experience with the latest design.

§ 2. Changes from R0

- Updated to reflect changes in P1030R7.
- Provided more data on usage experience to [Implementation and usage experience](#).
- Added a benchmark to [Performance](#).

§ 3. Problems

§ 3.1. Encoding

A significant portion of the initial revision of the paper ([\[P1030R0\]](#)) was devoted to examining the issues surrounding `std::filesystem::path` and the use of "ANSI encodings" (code pages - [\[WIN-CODEPAGE\]](#)) on Windows:

`std::filesystem` came originally from `Boost.Filesystem`, which in turn underwent three major revisions during the Boost peer review as it was such a lively debate. During those reviews, it was considered very important that paths were passed through, unmodified, to the system API. There are very good reasons for this, mainly that filesystems, for the most part, treat filenames as a bunch of bytes without interpreting them as anything. So any character reencoding could cause a path entered via copy-and-paste from the user to be unopenable, unless the bytes were passed through exactly.

This is a laudable aim, and it is preserved in this path view proposal. Unfortunately it has a most unfortunate side effect: on Microsoft Windows, `std::filesystem::path` when supplied with `char` not `wchar_t`, is considered to be in *ANSI* encoding. This is because the `char` accepting syscalls on Microsoft Windows consume ANSI for compatibility with Windows 3.1, and they simply thunk through to the UTF-16 accepting syscall after allocating a buffer and copying the input bytes into shorts. Therefore on Microsoft Windows, `std::filesystem::path` duly expands `char` input into its internal UTF-16 `wchar_t` storage via direct casting. It does not perform a UTF-8 to UTF-16 conversion.

Unfortunately any Microsoft Windows IDE or text editor that I have used recently defaults to creating C++ source files in UTF-8, exactly the same as on every other major platform including Linux and MacOS. This in turn means that source code with a `char` string literal such as `"UTF♠stringΩliteral"` makes a UTF-8 `char` string, not an ANSI `char` string, which is consistent across all the major platforms. Thus, `std::filesystem::path`'s behaviour on Microsoft Windows is quite surprising: your portable program will not work. What works on all the other platforms, without issue, does not work on Microsoft Windows, for no obvious reason to the uninitiated.

This author can only speak from his own personal experience, but what he has found over many years of practice in writing portable code based on `std::filesystem::path` is that one ends up inevitably using preprocessor macros to emit `L"UTF♠stringΩliteral"` when `_WIN32` and `_UNICODE` are macro defined, and otherwise emit `"UTF♠stringΩliteral"`. The reason is simple: the same string literal, with merely a `L` or not prefix, works identically on all platforms, no locale induced surprises, because we know that string literals in UTF source code will be in some UTF-x format. The side effect is spamming your 'portable' program code with string literal wrapper macros as if we were still writing for MFC, and/or `#if defined(_WIN32) && defined(_UNICODE)` all over your code. I do not find this welcome.

R0 goes as far as to switch to UTF-8 as the default encoding for `path_view`:

I propose that when char strings are supplied as a path string literal, and if and only if a conversion is needed, that we interpret those chars as UTF-8.

I know that this is a breaking change from `std::filesystem::path`, but I would argue that `std::filesystem::path` needs to be similarly changed. UTF-8 source code is very, very commonplace now, much more so than even a few years ago, and it is extremely likely that almost all new C++ written will be in UTF-8. So best to change `std::filesystem::path` appropriately, and if that is too great a breaking change, then these proposed path views are ‘fixed’ instead.

While this revision confuses source and literal encoding and presents an overly ambitious solution, the problems described by the author are very real. In fact, they have worsened as UTF-8 adoption has increased on Windows, particularly with the ease of enabling UTF-8 via the `/utf-8` compiler flag in MSVC.

Working with certain parts of `std::filesystem::path` is very error-prone for the increasingly common case of literal encoding being UTF-8. Unfortunately, later revisions of P1030 not only dropped any attempt to address this problem but exacerbated it by adopting the legacy ANSI encoding throughout the API. Worse still, this encoding has been embedded in the internal representation, making it part of the ABI — a major regression compared to `std::filesystem::path`, where the use of ANSI encoding is far more limited and rightfully avoided in the internal representation.

[P2319], which was recently approved by SG16 with strong support, proposes to deprecate the most problematic (from the encoding standpoint) parts of `std::filesystem::path`. [P1030R7] does the opposite and massively increases the public API (and ABI) surface that relies on error-prone legacy code pages.

In addition to problems described in [P1030R0], the use of ANSI encoding makes it hard for `std::filesystem::path_view` to interoperate with modern facilities such as C++20 `std::format` and C++23 `std::print` (see [Formatting](#)).

Handling of transcoding errors wasn’t specified up until revision R7 where it was introduced in an inconsistent manner and wasn’t approved or even seen by SG16.

§ 3.2. Implementation and usage experience

[P1030R7] claims:

If you wish to use an implementation right now, a highly-conforming reference implementation of the proposed path view can be found at

https://github.com/ned14/llfio/blob/master/include/llfio/v2.0/path_view.hpp.

Unfortunately, at the time of writing, important parts of the proposal are missing from that implementation. Specifically, more than 80 new overloads (for functions like `absolute` to `weakly_canonical`) remain unimplemented. Even worse, the paper itself lacks wording for these functions:

Wording note: The definitions for the function declared in the synopsis above are not provided at this time. All of them delegate to the overload taking a `path`.

Additionally, there is no implementation of a *path-view-like* equivalent that was designed on-the-fly during one of the LEWG reviews. As a result, there is no way to evaluate the effects of switching to `path_view` in these functions on real-world user code.

As of November 2024, GitHub Search reports only 38 files using `llfio::path_view` in C++ files, about half of which are in llfio itself or its forks. This suggests that usage experience with this implementation, even in its current form not fully matching the paper, is minimal. For comparison, there are 144 thousand results for `boost::filesystem::path` despite the latter being largely superseded by its standard counterpart.

The only notable open-source project that we could find that considered using `llfio::path_view` is the Nix package manager ([NIX-ISSUE9205]). However, they went with a different design that doesn't exhibit encoding, performance and complexity problems of P1030.

§ 3.3. Performance

`path_view` in its current form exacerbates encoding problems, but does it at least offer performance improvements?

Unfortunately, `path_view` goes to great lengths to avoid providing any performance benefits for existing users. This is achieved through obscure *path-view-like* overloads so that

existing C++ code would need to ‘opt in’ to using the path view overloads

This stands in stark contrast to the common use of `std::string_view`, which typically allows users to avoid `std::string` allocations:

```
void f(std::string_view s);

f("foo"); // No allocation

std::filesystem::file_size("/path/to/file"); // Allocates std::filesystem::
                                              // in P1030R7.
```

Additionally, due to lazy transcoding, `std::filesystem::path_view` can be slower than `std::filesystem::path`, which transcodes eagerly, when used multiple times.

[P1030R7] doesn't say much about performance, just hints at avoiding memory allocations in some cases. Since there is no implementation provided for most of the APIs it is hard to evaluate it. So we implemented a small subset of the API based on specification, however vague, and benchmarked it instead:

```
#include <benchmark/benchmark.h>
#include <fmt/format.h>
#include <llfio/llfio.hpp>

namespace llfio = llfio_v2_b1279174;

uintmax_t file_size_impl(const char *p, std::error_code *ec = nullptr);

namespace fs {

using llfio::path_view;

struct path_view_like {
    path_view view;

    template <typename T, std::enable_if_t<std::is_convertible_v<T, path_view>
                                     || !std::is_convertible_v<T, std::string_view>
                                     || std::is_convertible_v<T, int> == 0>
    path_view_like(const T &p) : view(p) {}
};

using std::filesystem::file_size;
```

```

uintmax_t file_size(path_view_like p) {
    return file_size(p.view.path());
}

uintmax_t file_size(path_view_like p, std::error_code &ec) noexcept {
    return file_size(p.view.path(), ec);
}

uintmax_t file_size_optimized(path_view_like p, std::error_code *ec = nullptr) {
    return file_size_impl(p.view.render_zero_terminated(p.view).c_str(), ec);
}

} // namespace fs

class fast_path {
private:
    fmt::basic_memory_buffer<char, PATH_MAX> buf_;

public:
    fast_path(const char *p) {
        auto len = strlen(p);
        buf_.resize(len + 1);
        strcpy(buf_.data(), p);
    }

    const char *c_str() const { return buf_.data(); }
};

inline uintmax_t file_size_optimized(const fast_path &p) {
    return file_size_impl(p.c_str());
}

const char *filename = __FILE__;

static void path(benchmark::State &state) {
    for (auto _ : state) {
        std::filesystem::file_size(std::filesystem::path(filename));
    }
}

static void path_optimized(benchmark::State &state) {
    for (auto _ : state) {
        file_size_optimized(fast_path(filename));
    }
}

```

```

static void path_view(benchmark::State &state) {
    for (auto _ : state) {
        fs::file_size(fs::path_view(filename));
    }
}

static void path_view_optimized(benchmark::State &state) {
    for (auto _ : state) {
        fs::file_size_optimized(fs::path_view(filename));
    }
}

static void native_string(benchmark::State &state) {
    for (auto _ : state) {
        file_size_impl(filename);
    }
}

BENCHMARK(path);
BENCHMARK(path_optimized);
BENCHMARK(path_view);
BENCHMARK(path_view_optimized);
BENCHMARK(native_string);

BENCHMARK_MAIN();

```

`file_size_impl` is an implementation of `file_size` taken from libc++.

Results on macOS compiled with Apple clang version 15.0.0 (clang-1500.3.9.4):

```

Unable to determine clock rate from sysctl: hw.cpufrequency: No such file or directory
This does not affect benchmark measurements, only the metadata output.
***WARNING*** Failed to set thread affinity. Estimated CPU frequency may be off.
2024-07-07T11:42:23-07:00
Running ./path-view-test
Run on (8 X 24 MHz CPU s)
CPU Caches:
  L1 Data 64 KiB
  L1 Instruction 128 KiB
  L2 Unified 4096 KiB (x8)
Load Average: 1.94, 2.56, 2.58

```

Benchmark	Time	CPU	Iterations
<code>path</code>	812 ns	810 ns	717698
<code>path_optimized</code>	765 ns	764 ns	910143
<code>path_view</code>	812 ns	809 ns	869576
<code>path_view_optimized</code>	780 ns	779 ns	899396
<code>native_string</code>	749 ns	748 ns	928123

`path_view` uses the implementation suggested in the wording and, as expected, has the same performance as the `path` overload it forwards to. `path_view_optimized` avoids constructing `path` and gives a minor improvement of ~4%. `path_optimized` demonstrates that even bigger improvement can be achieved without any complexity of `path_view` just by providing an API-compatible version of `path` with a larger inline buffer. It gives ~6% improvement.

`path_view` as specified in R7 is inherently slower than a `path(view)` implementation that uses a single representation because of an additional runtime dispatch.

§ 3.4. Formatting and output

Unlike `std::filesystem::path`, `std::filesystem::path_view` proposed by the paper did not provide a formatter until R7 published in September 2024.

Sadly, the newly added formatter is not fully specified and even its current partial specification has major problems due to unfortunate choices in the latest design.

One issue is related to encoding. The representation of `path` uses a single encoding that remains constant at runtime, making it feasible — though not trivial — to specify a good formatter. In contrast, `path_view` complicates matters by using multiple representations with different encodings, one of which can be a legacy encoding that can change at runtime. As a result, there is no way to determine which encoding `path_view` was constructed with at the time of use. This is conceptually similar to the Time of Check to Time of Use ([TOCTOU]) class of problems common in filesystem operations, which in this case can lead to mojibake, data corruption and other problems.

Another issue is the binary representation, which is severely underspecified and may conflict with other representations, making output hard or impossible to round-trip, even within a single implementation. Writing as an author of the `path` formatter ([P2845]), it remains unclear from [P1030R7] how it is expected to work.

`operator<<` is defined in terms of *path-from-binary* which appears to have the same problems.

§ 3.5. Complexity

The proposed `std::filesystem::path_view` roughly doubles the API surface area of `std::filesystem::path`, both in terms of its own definition and by proposing to add an overload that takes *path-view-like* arguments for every existing overload that takes `path`. For example:

```
bool equivalent(const path& p1, const path& p2);  
bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;  
  
bool equivalent(path-view-like p1, path-view-like p2);  
bool equivalent(path-view-like p1, path-view-like p2, error_code& ec)noexcept;
```

Contrary to its name, the proposed `std::filesystem::path_view` is not truly a view of `std::filesystem::path` in the same way that `std::string_view` can be considered a view of `std::string`. `path` has a single representation that is suitable for the current system. In contrast, `path_view` is effectively a discriminated union of some (but not all) of the types from which `path` can be constructed, with a lazy conversion to `path`. To further complicate things, `path_view` is also constructible from inputs, `path` is not constructible from. It is unclear what such an unusual API should be called, but it probably should not be referred to as a "view."

§ 3.6. Conclusion

In summary, the proposed `std::filesystem::path_view` presents significant concerns that need to be resolved before standardization. Its design exacerbates encoding problems and adds unnecessary complexity to the API. The reliance on legacy code pages undermines modern practices and complicates interoperability with other C++ facilities.

Additionally, the increased API surface area and the requirement for users to opt in to specific overloads detract from its usability. To maximize the utility of `path_view`, future revisions should focus on simplifying its design, addressing encoding issues, enhancing compatibility with existing libraries and getting actual implementation and usage experience. Standardizing the current proposal risks introducing more problems than it solves.

§ References

§ Informative References

[NIX-ISSUE9205]

Use `std::filesystem::path` for `Path`. URL: <https://github.com/NixOS/nix/issues/9205>

[P1030R0]

Niall Douglas. *P1030R0: `std::filesystem::path_view`*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1030r0.pdf>

[P1030R7]

Niall Douglas. *P1030R7: `std::filesystem::path_view`*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1030r7.pdf>

[P2319]

Victor Zverovich. *Prevent path presentation problems*. URL: <https://wg21.link/p2319>

[P2845]

Victor Zverovich. *Formatting of `std::filesystem::path`*. URL: <https://wg21.link/p2845>

[TOCTOU]

Time-of-check to time-of-use. URL: https://en.wikipedia.org/wiki/Time-of-check_to_time-of-use

[WIN-CODEPAGE]

Windows code page. URL: https://en.wikipedia.org/wiki/Windows_code_page